OBJECT ORIENTED PROGRAMMING USING C++

MINI PROJECT REPORT

Project Title: Cyber-Forensic Integrity Suite

Submitted by

Group ID: CSF-GRP-04

PROJECT TEAM MEMBERS

Sr. No.	PRN Number	Student Name
1	1262250194	Pranav Sahoo
2	1262251830	Tanushree Bali
3	1262250907	Arun S
4	1262251697	Neel Mehta

Under the Guidance of

Prof. Mugdha Joshi

Department of Computer Engineering and Technology

Dr. Vishwanath Karad MIT World Peace University, Pune – 411 038

Maharashtra, India

Academic Year 2026–2027

DEPARTMENT OF COMPUTER ENGINEERING AND TECHNOLOGY

CERTIFICATE

This is to certify that

1262250194 Pranav Sahoo

1262251830 Tanushree Bali

1262250907 Arun S

1262251697 Neel Mehta

students of Second Year B. Tech. Computer Science and Engineering (Cybersecurity and Forensics), have completed the mini project entitled **"Cyber-Forensic Integrity Suite"** and submitted this report as part of the requirements for the course **CSF20210: Object Oriented Programming Laboratory** during the academic year 2026–2027.

Date: 23/10/2026

Place: Pune

Prof. Mugdha Joshi
Course Teacher
Department of Computer Engg. and Tech.
MIT WPU, Pune

ACKNOWLEDGEMENT

We express our profound gratitude to our esteemed guide Dr. Vinayak Musale, Department of Computer Engineering and Technology, for his continuous encouragement, expert guidance, and valuable insights throughout the development of this mini project. We are also deeply thankful to Dr. Vishwanath Karad MIT World Peace University, Pune, for providing exceptional academic infrastructure and laboratory facilities. Finally, we thank our peers and family members who provided technical support and constant motivation.

Student Name(s)

Pranav Sahoo, Tanushree Bali, Arun S, Neel Mehta

CSF-GRP-04

Date: 23/10/2026

ABSTRACT

Digital evidence integrity is paramount in cybersecurity investigations, ensuring that forensic artifacts remain tamper-free and legally admissible. This project, titled 'Cyber-Forensic Integrity Suite', is a robust C++ console application designed to verify digital evidence integrity through checksum hashing, anchored on core object-oriented programming (OOP) principles. The system implements three core classes (Artifact, ForensicsEngine, and LiveLogAnalyzer), applying eight foundational OOP concepts including encapsulation, abstraction, inheritance, polymorphism, operator overloading (==), friend functions, static members, and dynamic memory management. The application ingests digital evidence designations, computes deterministic checksums, compares live states against trusted baselines, and generates detailed forensic audit dossiers. Through modular design and rigorous testing against simulated alterations, the suite successfully flags unauthorized modifications with instant [SECURE] or [CRITICAL] verdicts. The project bridges theoretical OOP paradigms with practical cybersecurity forensic workflows.

Keywords: Digital Forensics, C++, Object-Oriented Programming, Checksum Verification, Integrity Audit, Cybersecurity.

TABLE OF CONTENTS

Certificate
Acknowledgement
Abstract
List of Figures
List of Tables (if applicable)
List of Abbreviations (if applicable)

Sr. No. Chapter Page No.

1. Introduction 6
   1.1 Motivation 6
   1.2 Objectives 6
2. Problem Definition 7
3. Software / Hardware Requirements, Tools and Technologies 8
4. System Design and Architecture, Class Diagrams / Flow Chart Design 9
5. Implementation and User Interface 10
6. Methodology 12
7. Algorithms and Testing 13
8. Conclusion and Future Scope 14
   References 15

CHAPTER 1

INTRODUCTION

1.1 Background and Motivation

In modern cybersecurity and digital forensics investigations, maintaining the uncompromised integrity of digital artifacts (such as firewall logs, system dumps, and configuration files) is critical. Evidence files are highly susceptible to silent tampering, accidental corruption, or unauthorized swapping during acquisition and analysis. Traditional manual verification is error-prone and lacks traceability. The motivation behind this project is to build an automated, reliable, and object-oriented forensic suite that instantly fingerprints digital evidence using checksums, establishes immutable security baselines, and provides verifiable audit dossiers to aid forensic investigators.

1.2 Problem Statement

Digital evidence files can be altered, swapped, or corrupted without any visible signs on the storage media. Investigating officers and security analysts require a robust mechanism to ingest target evidence assets, compute deterministic verification signatures (checksums), and detect any unauthorized discrepancies or tampering between baseline evidence and live operational states in real time.

1.3 Objectives

1. To develop a C++ console application implementing core Object-Oriented Programming (OOP) principles for forensic evidence analysis.
2. To implement a deterministic checksum generation and verification engine (ASCII-based weighting formula).
3. To establish abstract base classes and derived analyzers (ForensicsEngine and LiveLogAnalyzer) utilizing runtime polymorphism.
4. To incorporate operator overloading (==) for intuitive signature comparisons and friend functions for generating secure audit dossiers.
5. To provide interactive terminal workflows supporting live integrity scans, anomaly detection, and session metrics tracking.

1.4 Scope and Limitations

Scope: The suite covers digital artifact ingestion, baseline signature establishment, live integrity scanning ([SECURE] vs [CRITICAL]), equality operator testing, session metrics tracking, and formatted audit dossier export.
Limitations: The current checksum formula computes signatures based on the artifact filename/string rather than reading actual underlying file binary bytes. Additionally, inputs are read via standard cin, so filenames containing spaces are not fully supported without tokenization.

1.5 Report Organisation

Chapter 1 introduces the project background, motivation, objectives, and scope. Chapter 2 details the problem definition and functional requirements. Chapter 3 outlines software and hardware requirements. Chapter 4 presents system architecture, class diagrams, and component interactions. Chapter 5 covers implementation details, module descriptions, and execution screenshots. Chapter 6 discusses development methodology. Chapter 7 presents algorithms and test cases. Chapter 8 concludes with future enhancements.

CHAPTER 2

PROBLEM DEFINITION AND OBJECTIVES

2.1 Problem Definition

Digital forensics teams frequently handle large volumes of log files and system dumps. Ensuring that an evidence file analyzed in court is identical to the artifact seized at the crime scene requires cryptographic or mathematical integrity verification. Without automated verification tools, forensic audits are vulnerable to human oversight and malicious tampering.

2.2 Existing System and Its Limitations

Existing approaches rely on manual file inspection or external command-line utilities (such as standard sha256sum or md5sum tools) which are decoupled from application logic, lacking object-oriented modularity, integrated session tracking, and structured forensic dossiers.

2.3 Proposed Solution

The Cyber-Forensic Integrity Suite provides an integrated object-oriented environment where digital artifacts are encapsulated into Artifact objects. It computes deterministic checksums, executes polymorphism-driven live scans, supports operator overloading for direct signature comparison, and generates secure forensic audit dossiers via privileged friend functions.

2.4 Functional and Non-Functional Requirements

Functional Requirements:
- Ingest target evidence filenames and compute numeric checksums.
- Execute live integrity scans comparing live states against established baselines (returning SECURE or CRITICAL verdicts).
- Overload equality operator (==) to compare artifact signatures.
- Generate and display formal forensic audit dossiers.
- Track and display total digital artifacts processed in the session.

Non-Functional Requirements:
- Reliability: Deterministic calculations ensure consistent verification.
- Maintainability: Clean object-oriented architecture adhering to SOLID principles.
- Usability: Interactive console menu with clear prompts and status outputs.

CHAPTER 3

REQUIREMENTS, TOOLS AND TECHNOLOGIES

3.1 Hardware Requirements

- Processor: Intel Core i5 / AMD Ryzen 5 or higher
- RAM: 8 GB or higher (32 GB supported on development Lenovo Yoga system)
- Storage: 50 MB available disk space

3.2 Software Requirements

- Operating System: Windows 11 / Linux (Ubuntu 22.04 LTS)
- Compiler: GCC / MinGW C++17 compliant compiler
- IDE / Editor: PyCharm / Visual Studio Code / CLion

3.3 Programming Language and Technologies

The project is implemented entirely in standard C++ (C++17), leveraging object-oriented design, STL containers (std::string, iostream), dynamic memory allocation (new/delete), virtual functions, operator overloading, and friend functions.

3.4 Development Environment

Developed and tested on a Lenovo Yoga laptop (32 GB RAM, Intel Arc graphics) running Windows 11, compiled via GNU G++ compiler with standard terminal execution.

CHAPTER 4

SYSTEM DESIGN AND ARCHITECTURE

4.1 System Architecture

The system architecture follows a modular, object-oriented design comprising core data classes, inheritance hierarchies, and procedural menu controllers. The Artifact class encapsulates evidence metadata, while the ForensicsEngine abstract base class drives polymorphic scan execution through derived LiveLogAnalyzer instances.

4.2 Use-Case / Flowchart / Data-Flow Diagram

The workflow initiates with banner display and baseline evidence designation input. The user is presented with an interactive menu loop offering 5 operations: (1) Run Live Integrity Scan, (2) Test Overloaded Operator, (3) Export Forensic Audit Dossier, (4) View Session Statistics, and (5) Terminate Operation.

4.3 Class Diagram and Object-Oriented Design

Key Classes and Relationships:
- Artifact: Encapsulates filename (string) and checksum (int). Maintains static int totalScanned. Provides computeChecksum(), const getters, overloaded operator==, and a friend function generateAuditTrail().
- ForensicsEngine: Abstract base class featuring pure virtual function executeScan() = 0 and a virtual destructor.
- LiveLogAnalyzer: Publicly inherits from ForensicsEngine, composing an Artifact object (has-a relationship) and storing baselineHash to execute polymorphic integrity inspections.

4.4 Data Design (if applicable)

Data structures utilize encapsulation and private member variables accessible only through const member functions and authorized friend functions.

CHAPTER 5

IMPLEMENTATION AND USER INTERFACE

5.1 Module Description

1. Ingestion Module: Prompts for target evidence designation, instantiates baseline Artifact objects, and increments static counters.
2. Live Scan Module: Dynamically allocates LiveLogAnalyzer polymorphic engines to compare live inputs against baseline checksums, outputting [SECURE] or [CRITICAL] verdicts.
3. Operator Overload Module: Tests equality between baseline and secondary artifacts using overloaded operator==.
4. Audit Dossier Module: Utilizes a privileged friend function generateAuditTrail() to output formatted forensic audit dossiers.
5. Statistics Module: Reports total digital artifacts processed via static member functions.

5.2 Key Implementation Details

Implementation utilizes C++ class encapsulation, constructor initializer lists, const correctness, dynamic polymorphic binding via base class pointers, and safe heap memory management with new and delete.

5.3 User Interface / Console Screenshots

The application provides an interactive, terminal-based user interface featuring clean banner headers, distinct teal prompts, and structured audit logs. The execution screenshots demonstrate four core operational phases:

Figure 5.1: Live Integrity Inspection Output
The user inputs 'server_log.txt' as baseline evidence. Upon running option 1 and testing against an altered state file 'server_data.txt', the polymorphic engine initiates a deep integrity inspection and immediately triggers a [CRITICAL] Status Red warning due to a checksum mismatch.

Figure 5.2: Operator Overload Hash Comparison
Selecting option 2 prompts for a secondary artifact ('stack_data.bin'). The overloaded equality operator (==) evaluates the checksums and outputs '[MISMATCH] Operator Overload: Signatures differ entirely.', verifying the robustness of operator overloading.

Figure 5.3: Exporting Forensic Audit Dossier
Selecting option 3 invokes the privileged friend function `generateAuditTrail()`, directly accessing private class members to export a formal Forensic Audit Dossier displaying the Target Asset ('server_log.txt') and Crypto Hash signature ([10377]).

Figure 5.4: Session Statistics Tracking
Selecting option 4 invokes static member function `displayTotalScans()`, displaying the total digital artifacts processed during the session (3).

5.4 Object-Oriented Programming Concepts Used

1. Encapsulation: Private data members (filename, checksum) in Artifact, exposed strictly through const getters.
2. Abstraction: ForensicsEngine hides implementation details behind pure virtual interface executeScan().

3. Inheritance: LiveLogAnalyzer publicly inherits from ForensicsEngine.
4. Polymorphism: Virtual executeScan() invoked through base-class pointers resolved at runtime.
5. Composition: LiveLogAnalyzer contains an Artifact targetEvidence object.
6. Operator Overloading: operator== compares two Artifact objects by checksum.
7. Friend Function: generateAuditTrail() accesses private class members directly.
8. Static Member: totalScanned tracks total artifacts created across all instances.

5.5 Sample Input and Output

Input: Target Evidence -> firewall_log.txt
Live Scan Input -> firewall_log.bak
Output: [CRITICAL] Status Red: Tampering identified! Checksum mismatch.

CHAPTER 6

METHODOLOGY

6.1 Development Approach

An iterative object-oriented development approach was followed, starting from class design and checksum logic formulation, followed by inheritance hierarchy implementation, operator overloading, friend function integration, and rigorous console testing.

6.2 Workflow / Process

1. Launch Suite -> Display Header Banner.
2. Ingest Baseline Evidence -> Compute Checksum -> Increment Counter.
3. Enter Investigation Terminal Loop -> Select Menu Option (1-5).
4. Execute Desired Forensic Operation (Scan / Compare / Dossier / Stats).
5. Terminate Operation -> Exit Suite.

6.3 Module Integration

Modules are tightly integrated through object composition and polymorphic pointer invocation in main(), ensuring seamless data flow between artifact ingestion, integrity analysis, and audit reporting.

CHAPTER 7

ALGORITHMS AND TESTING

7.1 Algorithms / Pseudocode

Algorithm 1: Checksum Calculation
Input: string str
Output: int checksum
1. Initialize acc = 0
2. For each character c in str: acc += ASCII(c)
3. Return (acc * 7 + 31)

7.2 Test Plan and Test Cases

- Test ID T1: Baseline Ingestion. Input: firewall_log.txt. Expected: Evidence ingested, checksum computed. Actual: Success. Status: Pass.
- Test ID T2: Live Integrity Scan (Match). Input: firewall_log.txt vs firewall_log.txt. Expected: [SECURE] Status Green. Actual: Success. Status: Pass.
- Test ID T3: Live Integrity Scan (Mismatch). Input: firewall_log.txt vs firewall_log.bak. Expected: [CRITICAL] Status Red. Actual: Success. Status: Pass.
- Test ID T4: Operator Overload. Baseline == Secondary. Expected: Match/Mismatch verified. Actual: Success. Status: Pass.
- Test ID T5: Audit Dossier Export. Expected: Formatted dossier displayed. Actual: Success. Status: Pass.

7.3 Results and Discussion

The Cyber-Forensic Integrity Suite successfully met all stated objectives. All eight OOP concepts were seamlessly integrated into a compact, functional console application. Test results confirm accurate detection of file alterations and robust audit reporting.

CHAPTER 8

CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

The Cyber-Forensic Integrity Suite successfully achieves its goal of verifying digital evidence integrity using checksums built upon core object-oriented programming principles in C++. The project demonstrates practical application of encapsulation, polymorphism, inheritance, operator overloading, and friend functions in a realistic cybersecurity context.

8.2 Limitations

Checksums are computed from filenames/strings rather than file binary contents. Filenames containing spaces are not parsed correctly via cin.

8.3 Future Scope

1. Hash real file binary contents using cryptographic algorithms such as SHA-256.
2. Implement persistent file logging for forensic audit dossiers.
3. Expand inheritance hierarchy with additional specialized scanner classes.
4. Replace raw pointers with C++ smart pointers (std::unique_ptr).

REFERENCES

[1] B. Stroustrup, The C++ Programming Language, 4th ed. Boston: Addison-Wesley, 2013.
[2] B. Carrier, File System Forensic Analysis. Boston: Addison-Wesley Professional, 2005.
[3] K. Mandia, M. Prosise, and T. Pepe, Incident Response and Computer Forensics, 2nd ed. New York: McGraw-Hill, 2003.
[4] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software. Boston: Addison-Wesley, 1994.
[5] National Institute of Standards and Technology (NIST), "Guide to Integrating Forensic Techniques into Incident Response," NIST Special Publication 800-86, 2006.